APCS 程式設計系列 · U04 · 實作初級 ~ 中級 · Python版

一維陣列與前綴和 重點講義

陣列宣告與走訪 · 索引從 0 開始與越界 · 計數陣列 · 前綴和 · 雙指標與滑動視窗 · 反轉與平移

1. 陣列：一次存一整排資料

要「先全部讀完、再回頭比較」的題目，就必須把資料存起來。Python 用 **list**：一行 **list(map(int, input().split()))** 就把整排數字讀成串列，用 **a[i]** 存取，索引從 0 到 n-1。

Python 讀進陣列，算高於平均的人數

```python
n = int(input())
a = list(map(int, input().split()))   # a[0] ~ a[n-1]
total = sum(a)
cnt = 0
for x in a:
    if x * n > total:     # 不用除法，避免小數
        cnt += 1
print(cnt)
```

輸入	輸出
5 60 75 90 40 85	3

想做的事	C++	Python
開 n 個 0	vector<int> a(n, 0);	a = [0] * n
長度	a.size()	len(a)
加到尾巴	a.push_back(x);	a.append(x)
最後一個	a[n - 1]	a[n - 1] 或 a[-1]
逐一走訪	for (int x : a)	for x in a:

2. 索引從 0 開始，小心越界

Python 找最大值和它的位置

```python
a = [7, 2, 9, 4, 9, 1]
pos = 0
for i in range(1, len(a)):
    if a[i] > a[pos]:     # 用 > 留下第一個最大值
        pos = i
print(pos, a[pos])
```

輸入	輸出
（不需要輸入）	2 9

位置 2 是「第 3 個數」。題目若要求「第幾個」（從 1 開始數），輸出時要記得加 1。

索引的三個基本功

- n 個元素的索引是 **0 ~ n-1**，沒有 a[n]。迴圈寫 i < n，不是 i <= n。
- 題目從 1 開始編號時，可以多開一格、直接用 a[1] ~ a[n]，比較不容易算錯。
- Python 的 a[-1] 是最後一個元素、a[-2] 是倒數第二個。

陷阱提醒 越界不一定會報錯

【錯】a[n] 會出現 IndexError；但 a[i - 1] 在 i = 0 時變成 a[-1]，不會報錯，只會「默默算錯」。

【錯】b = a 不是複製，b 和 a 是「同一個」串列，改 b 等於改 a。要複製請寫 b = a[:]。

3. 計數陣列：把「值」當成索引

Python 統計每個分數出現幾次

```python
n = int(input())
cnt = [0] * 101           # 分數 0~100，要開 101 格
for s in map(int, input().split()):
    cnt[s] += 1           # 分數本身當索引
best = 0
for v in range(1, 101):
    if cnt[v] > cnt[best]:
        best = v
print(best, cnt[best])
```

輸入	輸出
8 70 85 70 100 85 70 60 85	70 3

70 和 85 都出現 3 次。從小掃到大、用 > 比較，次數一樣時留下「比較小」的分數。

什麼時候用計數陣列

- 數值範圍不大（例如 0 ~ 10^6），要問「每個值出現幾次」「有沒有重複」「哪個最多」。
- 讀一個數只要 cnt[x]++ 一步，n 個數 O(n)；不用兩層迴圈互相比較的 O(n^2)。
- 計數完由小到大掃一遍，就等於把資料「排好序」了。

陷阱提醒

【錯】值最大是 100，陣列只開 100 格：cnt[100] 就越界了。最大值是 M，就要開 M + 1 格。

【錯】值有負數（例如 -50 ~ 50）不能直接當索引，要先加上位移：cnt[x + 50]++。

4. 前綴和：區間和一次減法就算完

i	0	1	2	3	4	5
a[i]	（不用）	3	1	4	1	5
p[i]	0	3	4	8	9	14

前綴和的定義

- **p[i] = a[1] + a[2] + … + a[i]**（前 i 個數的和），並規定 **p[0] = 0**。
- 建表只要一個迴圈：p[i] = p[i-1] + a[i]。
- 第 l 個到第 r 個的和 = **p[r] - p[l-1]**。例如 a[2] ~ a[4] 的和 = p[4] - p[1] = 9 - 3 = 6。

Python 前綴和：預處理一次，每次查詢 O(1)

```python
n, q = map(int, input().split())
a = list(map(int, input().split()))
p = [0] * (n + 1)                # p[0] = 0
for i in range(1, n + 1):
    p[i] = p[i - 1] + a[i - 1]   # a 從 0 開始編號
for _ in range(q):
    l, r = map(int, input().split())
    print(p[r] - p[l - 1])
```

輸入	輸出
5 2 3 1 4 1 5 1 3 2 4	8 6

做法	預處理	每次查詢	q 次查詢總共
每次從 l 加到 r（暴力）	不用	O(n)	O(n × q)
前綴和	O(n)	O(1)	O(n + q)

陷阱提醒 前綴和的三個坑

【錯】寫成 p[r] - p[l]，會把第 l 個數也扣掉。減的是 **p[l-1]**。

【錯】查詢有 2×10^5 行時，一直呼叫 input() 太慢，改用 sys.stdin.read().split() 一次讀取。

【錯】Python 串列從 0 開始、題目從 1 開始編號時，a[i - 1] 和 p[i] 差一格，先用小例子對一次。

- n、q 都是 2×10^5 時，暴力要 4×10^{10} 次運算，一定超時；前綴和只要約 4×10^5 次。
- 看到「很多次詢問區間總和」，第一個想到的就是前綴和。

5. 雙指標與滑動視窗：$O(n^2)$ 變 O(n)

Python 連續 k 個數的最大和（滑動視窗）

```python
a = [2, -1, 3, 5, -2, 4, 1]
n, k = len(a), 3
s = sum(a[:k])           # 第一個視窗
best = s
for i in range(k, n):
    s += a[i] - a[i - k]   # 進來一個、出去一個
    best = max(best, s)
print(best)
```

輸入	輸出
（不需要輸入）	7

五個視窗的和依序是 4、7、6、7、3。每滑一格只做一次加減，整題 O(n)。

Python 排序好的陣列：兩數和等於 t（雙指標）

```python
a = [1, 3, 4, 6, 8, 11]
i, j, t = 0, len(a) - 1, 14
while i < j:
    s = a[i] + a[j]
    if s == t:
        break
    if s < t:
        i += 1          # 太小：左邊往右，讓和變大
    else:
        j -= 1          # 太大：右邊往左，讓和變小
print(a[i], a[j])
```

輸入	輸出
（不需要輸入）	3 11

兩種技巧的共同點

- **滑動視窗**：長度固定的連續區段，往右滑一格 = 加進新的一個、扣掉最舊的一個。
- **雙指標**：陣列要先排好序；i 從左、j 從右，每一步一定有一個指標往中間走，最多走 n 步。
- 兩層迴圈 $O(n^2)$ 在 $n = 10^5$ 會超時；這兩招都是 $O(n)$。

陷阱提醒

【錯】最大和的初始值設成 0：如果每個數都是負的，答案會錯印成 0。用「第一個視窗」當初始值。
【錯】雙指標用在「沒排序」的陣列上會漏掉答案，要先排序。

6. 陣列反轉與平移

Python 原地反轉與向左平移 k 格

```python
a = [1, 2, 3, 4, 5, 6]
n, k = len(a), 2
i, j = 0, n - 1
while i < j:
    a[i], a[j] = a[j], a[i]    # 頭尾交換，走到中間就停
    i += 1
    j -= 1
b = [a[(i + k) % n] for i in range(n)]    # 向左平移 k 格
print(*b)
```

輸入	輸出
（不需要輸入）	4 3 2 1 6 5

反轉後是 6 5 4 3 2 1，再向左平移 2 格：最前面的 6、5 繞到最後面。

陷阱提醒 反轉與平移的經典錯誤

【錯】交換的迴圈跑完整個陣列（i < n），每一對會被換兩次，結果又變回原樣。只能走到中間。
【錯】原地往右移一格寫成「由左往右」a[i] = a[i-1]，會把後面全部蓋成同一個數。要嘛另開陣列 b，要嘛由右往左做。
【錯】平移 k 格時 k 可能比 n 大，先取 k % n；往右平移相當於往左平移 n - k 格。

Python 的切片寫法

- 反轉：a[::-1]；向左平移 k 格：a[k:] + a[:k]。
- 切片會產生新的串列，考試看懂就好；實作時記得 k 要先 % n。

讀完這份，你應該要會

- □ 會用 list(map(int, input().split())) 存 n 個數，知道索引是 0 ~ n-1
- □ 知道 a[-1] 是最後一個、b = a 不是複製
- □ 會用計數陣列統計出現次數，知道值最大 M 要開 M + 1 格
- □ 會建前綴和，區間 [l, r] 的和寫得出 p[r] - p[l-1]
- □ 會用滑動視窗求連續 k 個的最大和，初始值不用 0
- □ 會原地反轉、會用 (i + k) % n 平移，也知道「由左往右覆蓋」的錯誤

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室

IG @jf_chuang

LINE 諮詢 jenfoo2021